

Relazione

Progetto di base di dati per il gestionale di uno zoo

Francesco Caravita – 0001161188
francesco.caravita2@studio.unibo.it
Luca Mularoni – 0001162772
luca.mularoni@studio.unibo.it

Contents

1	Analisi dei requisiti	2
1.1	Definizione delle specifiche in linguaggio naturale ed estrazione dei concetti principali	2
1.2	Intervista	3
1.3	Rilevamento delle ambiguità e correzioni proposte	4
2	Progettazione concettuale	6
2.1	Schema scheletro	6
2.2	Schema concettuale finale	10
3	Progettazione logica	11
3.1	Stima del volume dei dati	11
3.2	Descrizione delle operazioni principali e stima della loro frequenza	12
3.3	Schemi di navigazione e tabelle degli accessi	13
3.3.1	Schemi di navigazione	13
3.3.2	Tabella degli accessi	13
3.4	Raffinamento dello schema	14
3.4.1	Eliminazione di attributi composti e gerarchie	14
3.4.2	Scelta delle chiavi	14
3.5	Analisi delle ridondanze	15
3.6	Traduzione di entità e associazioni in relazioni	22
3.7	Schema relazionale finale	23
3.8	Traduzione delle operazioni in query SQL	24
3.8.1	Lettura	24
3.8.2	Inserimento	33
3.8.3	Update	36
3.8.4	Cancellazione	39
4	Progettazione dell'applicazione	42
4.1	Architettura dell'applicazione realizzata	42
4.2	Screenshot dell'interfaccia utente	43
4.2.1	Schermata principale	43
4.2.2	Schermata di login	44
4.2.3	Schermata dal punto di vista di admin	45
4.2.4	Schermata dal punto di vista del veterinario	46
4.2.5	Schermata dal punto di modifica di un animale (Admin)	47

Chapter 1

Analisi dei requisiti

1.1 Definizione delle specifiche in linguaggio naturale ed estrazione dei concetti principali

Si vuole realizzare un sistema informativo per la gestione di uno zoo, che permetta di organizzare, l'archivio degli animali, le attività interne, monitorare la situazione economica e gestire la vendita dei biglietti ai visitatori.

Lo zoo ospita numerosi animali di cui gestisce ogni aspetto, dalla pianificazione della dieta fino al monitoraggio dello stato di salute attraverso visite e trattamenti medici. Per ciascun animale il sistema memorizza informazioni anagrafiche quali nome, specie di appartenenza, età, sesso e stato di salute corrente. Ogni animale viene assegnato a un recinto specifico, che condivide con altri esemplari della medesima specie. I recinti sono identificati da un codice univoco e descritti da una tipologia (ad esempio acquatico, terrestre o tropicale), da una capienza massima da rispettare e dall'area dello zoo in cui sono collocati. Un recinto può ospitare più animali contemporaneamente, a condizione che non venga superata la capienza stabilita.

Per quanto riguarda la gestione del personale, si vogliono memorizzare i dati anagrafici dei dipendenti, il ruolo svolto (ad esempio veterinario, custode, guida) e i contatti. I dipendenti vengono assegnati a turni di lavoro organizzati su base giornaliera o settimanale; per ogni turno si registrano il dipendente assegnato, l'orario di inizio e fine e l'area dello zoo in cui presta servizio. Un dipendente può avere più turni, ma non può essere assegnato a più turni contemporaneamente nello stesso intervallo orario.

Lo zoo offre ai visitatori diverse tipologie di biglietto, tra cui biglietto standard, ridotto e speciale (ad esempio comprensivo di visita guidata). Per ogni tipologia si vogliono memorizzare prezzo e descrizione, mantenendo eventualmente uno storico delle variazioni di prezzo nel tempo.

I visitatori possono acquistare uno o più biglietti tramite una singola prenotazione. Per ogni acquisto si registrano la data, la quantità di biglietti, la tipologia scelta e il prezzo totale. Nel caso di acquisti di gruppo, più biglietti possono essere associati alla stessa prenotazione. Il sistema consente di mantenere uno storico completo delle vendite, utile per analisi e statistiche.

Per quanto riguarda la gestione economica, il sistema tiene traccia anche delle spese dello zoo. Ogni spesa è caratterizzata da una data, un importo, una categoria (ad esempio alimentazione animali, manutenzione strutture, altre spese operative) e una descrizione opzionale. Questo permette di confrontare le entrate derivanti dalla vendita dei biglietti

con le uscite sostenute.

Il sistema deve consentire di registrare e aggiornare le informazioni relative ad animali, recinti, personale, turni di lavoro, vendite e spese. Inoltre, deve permettere la consultazione dello storico dei dati e la produzione di statistiche, come il numero di biglietti venduti in un certo periodo, il ricavo totale, il confronto tra entrate e uscite, i periodi di maggiore affluenza, i dipendenti con più turni assegnati e i recinti con il maggior numero di animali.

Infine, alcune operazioni sono riservate all'amministratore del sistema, che può gestire le specie animali, creare o eliminare recinti e aree, inserire e modificare i dati dei dipendenti, pianificare i turni di lavoro, definire le tipologie di biglietto e i relativi prezzi, nonché registrare e controllare le spese e accedere a tutte le statistiche globali del sistema.

Concetti principali estratti

I concetti centrali del dominio sono:

- **Animale, Specie, Habitat, Famiglia specie, Stato esistenza;**
- **Area, Recinto, Tipo area, Tipo recinto;**
- **Dipendente, Mansione, Turno, Stipendio storico;**
- **Visita medica;**
- **Utente con ruoli di accesso;**
- **Tipo biglietto, Scontrino, Dettaglio scontrino, Transazione, Categoria transazione;**
- **Fornitore, Tipo cibo, Fornitore cibo, Ordine giornaliero.**

1.2 Intervista

Di seguito è riportata una sintesi dell'intervista svolta, da cui sono emersi i requisiti fondamentali del sistema.

Domanda	Risposta / indicazione del committente
Quali categorie di utenti devono usare il sistema?	Il sistema è usato dal personale dello zoo con ruoli diversi: amministratore, cassiere, guardiano e veterinario. Alcune sezioni sono accessibili anche in consultazione da parte degli utenti pubblici.
Quali dati devono essere memorizzati?	Devono essere memorizzati animali, specie, habitat, famiglie tassonomiche, aree, recinti, dipendenti, turni, visite mediche, biglietti, scontrini, transazioni, ordini di cibo, fornitori e donazioni.
Quali operazioni devono essere più immediate?	Le operazioni più frequenti sono la vendita dei biglietti, la consultazione del saldo, la gestione degli ordini di cibo, l'inserimento delle visite mediche e la visualizzazione dell'anagrafica degli animali.

Domanda	Risposta / indicazione del committente
Esistono vincoli importanti?	Sì. Ogni animale appartiene a una sola specie, ogni recinto si trova in una sola area, ogni dipendente ha una mansione, ogni visita medica è svolta da un veterinario, e gli animali possono cambiare recinto nel tempo.
Serve tracciare la storia delle modifiche?	Sì, per alcuni aspetti è essenziale: collocazione degli animali, stipendio dei dipendenti, visite mediche, turni e movimenti economici devono essere storicizzati.

1.3 Rilevamento delle ambiguità e correzioni proposte

Dalla specifica iniziale emergono alcune ambiguità che sono state risolte in fase di progettazione.

Elemento ambiguo	Problema	Correzione adottata
Utente	Può essere confuso con un visitatore finale.	Nel database rappresenta un operatore autenticato del gestionale, associato a un dipendente e a un ruolo applicativo.
Stato esistenza	Il nome può far pensare allo stato dell'animale.	È usato per la classificazione della specie, ad esempio per indicare lo stato di conservazione secondo categorie come LC, VU, EN, CR.
Transazione e Scontrino	Rischiano di apparire ridondanti.	Lo scontrino rappresenta l'atto di vendita al cliente, mentre la transazione rappresenta il movimento contabile, che può essere di entrata o uscita.
Vivo e Data uscita	Potrebbero sembrare informazioni sovrapposte.	La coppia è mantenuta perché consente di distinguere tra animale presente, animale trasferito e animale deceduto, conservando la cronologia.
Storico collocazione	Senza una storia dei recinti si perderebbe l'evoluzione degli spostamenti.	Si introduce un'entità storica separata per registrare le permanenze nel tempo di ciascun animale nei recinti.

Elemento ambiguo	Problema	Correzione adottata
Ordine giornaliero	Può sembrare un generico acquisto.	È specializzato in ordini di cibo, collegato a fornitore, tipo di cibo e transazione economica associata.

Chapter 2

Progettazione concettuale

2.1 Schema scheletro

Lo schema scheletro contiene le entità principali del dominio: **Animale**, **Specie**, **Area**, **Recinto**, **Dipendente**, **Visita medica**, **Utente**, **Tipo biglietto**, **Scontrino**, **Transazione**, **Fornitore**, **Ordine giornaliero**, **Donazione**.

In questa fase le entità sono state introdotte nella forma più immediata, con una selezione minima di attributi e senza ancora esplicitare tutte le gerarchie di classificazione e le entità di supporto.

Lo schema concettuale finale è organizzato per aree funzionali.

Classificazione zoologica

Specie è collegata a **Stato esistenza**, **Habitat** e **Famiglia specie**. Ogni animale appartiene a una sola specie. [link mermaid live editor](#) [Schema concettuale Classificazione zoologica](#)

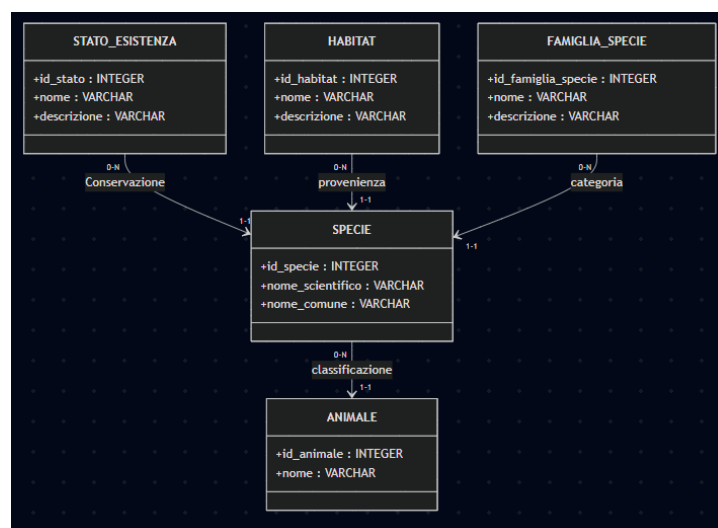


Figure 2.1: schema concettuale

Gestione degli spazi

Area contiene uno o più **Recinti**. Ogni recinto appartiene a una sola area e ha una tipologia. La presenza storica degli animali nei recinti è modellata tramite **Storico collocazione**. [link mermeid live editor](#) [Schema concettuale](#) [Gestione degli spazi](#)

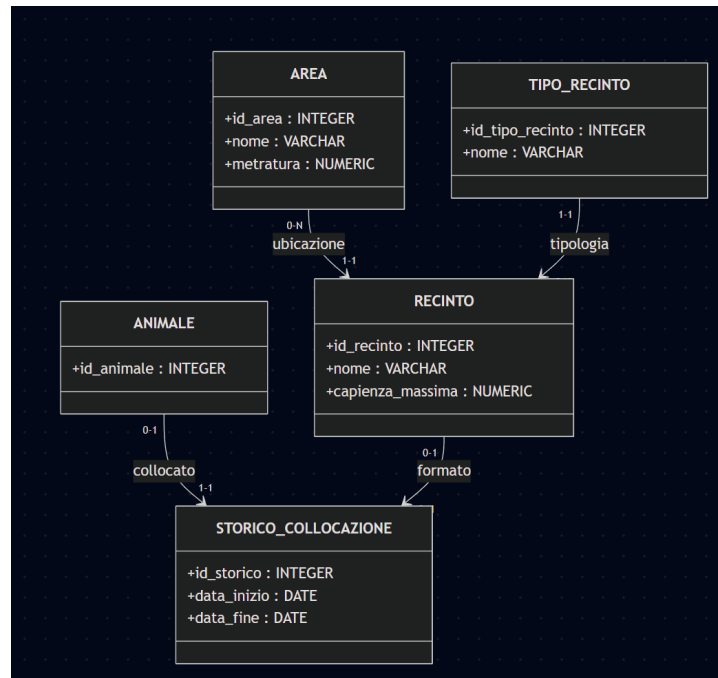


Figure 2.2: Schema concettuale

Gestione del personale

Ogni **Dipendente** ha una mansione. I turni vengono registrati tramite **Turno** e l'andamento degli stipendi è conservato da **Storico stipendio**. Un **Utente** del gestionale è collegato a un dipendente e a un ruolo applicativo. [link mermeid live editor](#) [Schema concettuale](#) [Gestione del personale](#)

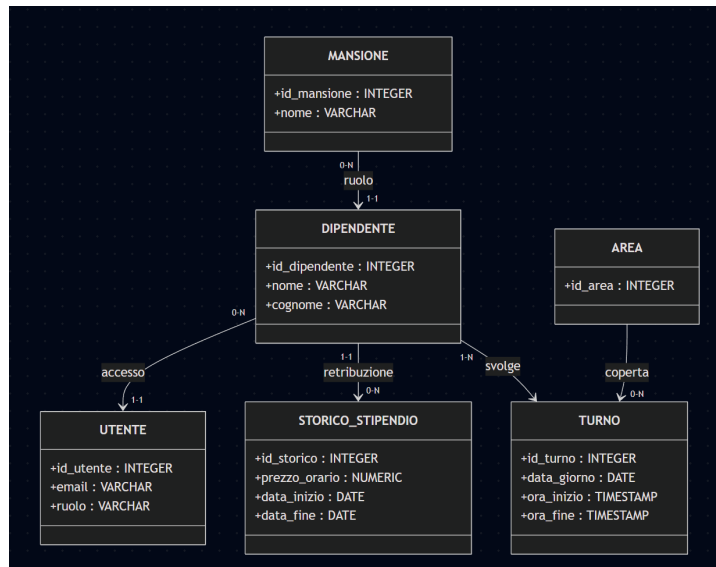


Figure 2.3: Schema concettuale

Sanità veterinaria

Le **Visite mediche** collegano animali e veterinari, con data di inizio, eventuale data di fine, diagnosi, peso e note terapeutiche. [link mermeid live editor Schema concettuale Sanità veterinaria](#)

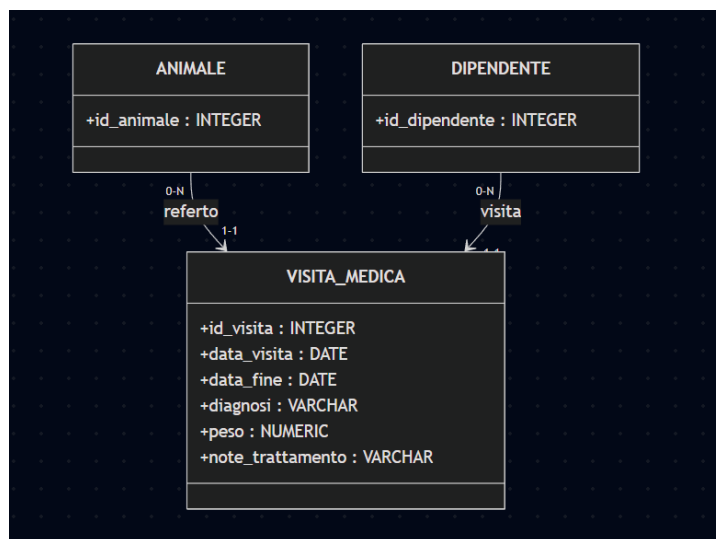


Figure 2.4: Schema concettuale

Vendita biglietti e contabilità

La biglietteria è gestita con **Tipo biglietto**, **Scontrino**, **Dettaglio scontrino** e **Transazione**. Le transazioni sono classificate in entrate o uscite mediante **Categoria transazione**. [link mermeid live editor Schema concettuale Vendita biglietti e contabilità](#)

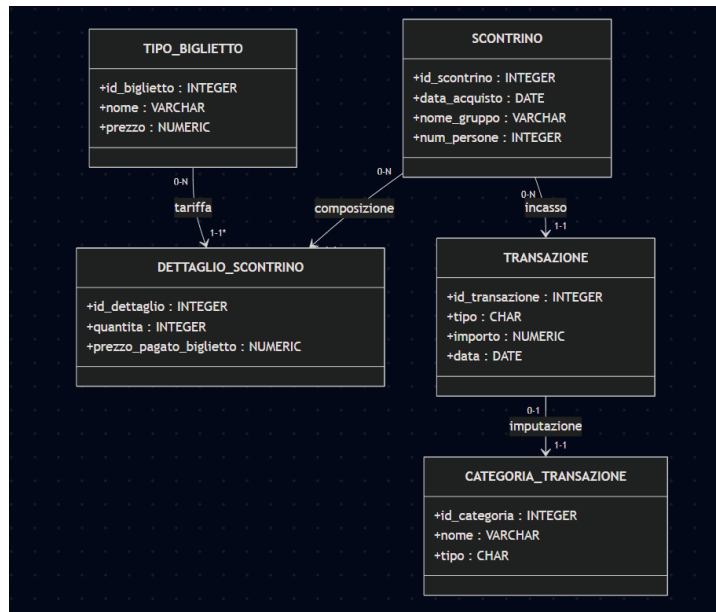


Figure 2.5: Schema concettuale

Approvvigionamenti

Gli ordini giornalieri di cibo vengono modellati con **Fornitore**, **Tipo cibo**, **Fornitore cibo** e **Ordine giornaliero**. La relazione tra fornitore e tipo di cibo permette di conoscere i prezzi disponibili. [link mermeid live editor](#) [Schema concettuale Approvvigionamenti](#)

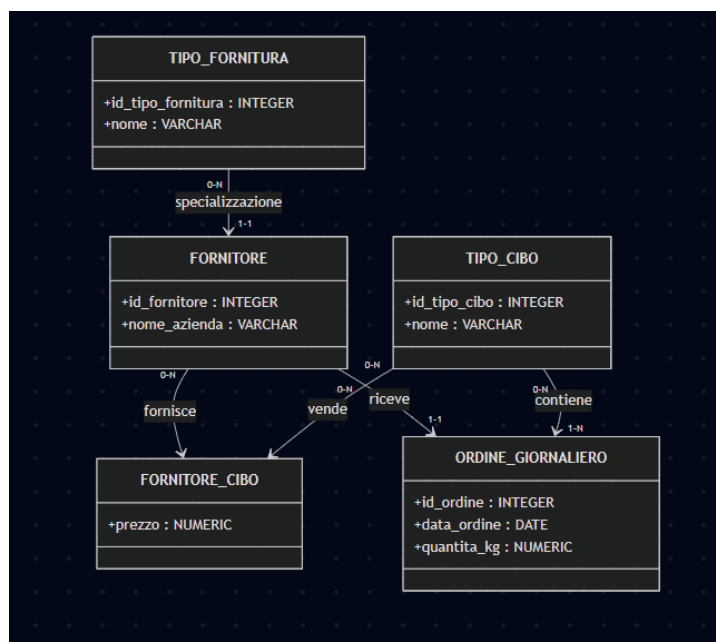


Figure 2.6: Schema concettuale

2.2 Schema concettuale finale

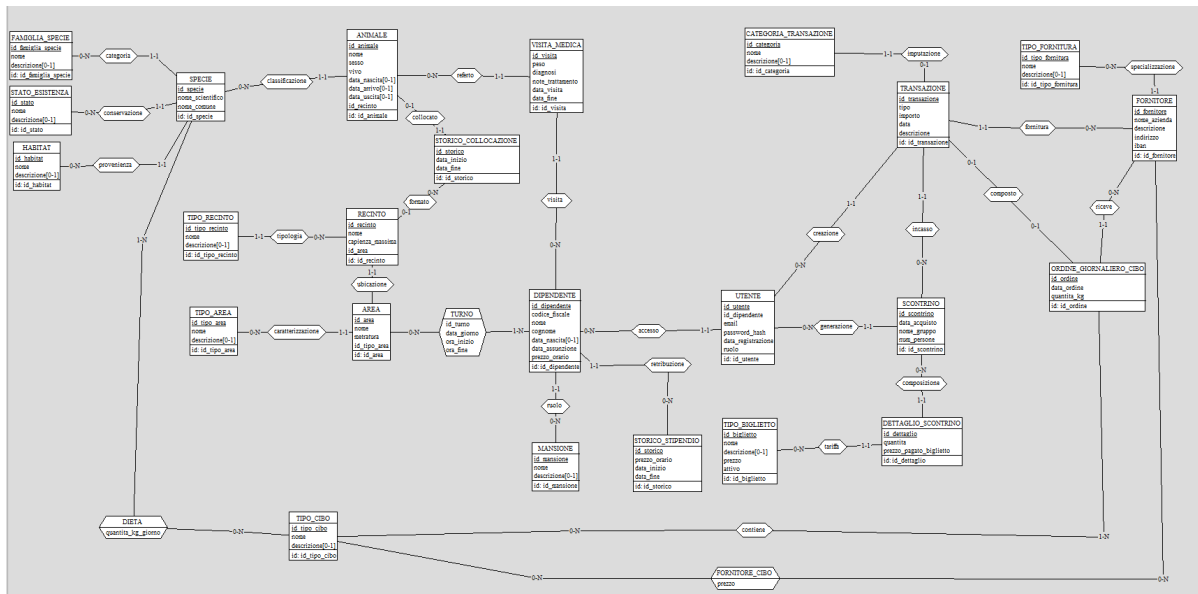


Figure 2.7: Schema concettuale finale

Chapter 3

Progettazione logica

3.1 Stima del volume dei dati

La tabella seguente riporta una stima ragionata del volume delle istanze per ciascuna entità/relazione. Le cardinalità sono coerenti con un zoo di dimensione medio-piccola, con un numero di transazioni e scontrini significativamente maggiore rispetto alle anagrafiche di base.

Elemento	Volume stimato
STATO_ESISTENZA	6
HABITAT	7
FAMIGLIA_SPECIE	6
TIPO_AREA	5
TIPO_RECINTO	5
MANSIONE	6
TIPO_CIBO	6
TIPO_FORNITURA	5
TIPO_BIGLIETTO	6
CATEGORIA_TRANSAZIONE	6
SPECIE	20
DIETA	40
AREA	20
RECINTO	40
ANIMALE	150
STORICO_COLLOCAZIONE	400
DIPENDENTE	40
STORICO_STIPENDIO	80
TURNO	1500
VISITA_MEDICA	500
UTENTE	40
FORNITORE	20
FORNITORE_CIBO	30
ORDINE_GIORNALIERO	8000
SCONTRINO	120000
DETTAGLIO_SCONTRINO	200000

Elemento	Volume stimato
TRANSAZIONE	150000

3.2 Descrizione delle operazioni principali e stima della loro frequenza

Le operazioni sono state suddivise in operazioni di consultazione e operazioni di manutenzione. Le prime sono eseguite frequentemente dagli operatori, le seconde più raramente dall'amministratore.

Operazione	Frequenza	Tabelle/relazioni principali
Vendita di un biglietto	300/giorno	TIPO_BIGLIETTO, SCONTRINO, DETTAGLIO_SCONTRINO, TRANSAZIONE
Consultazione saldo e ultime transazioni	50/giorno	TRANSAZIONE, CATEGORIA_TRANSAZIONE
Statistiche biglietteria	20/giorno	SCONTRINO, DETTAGLIO_SCONTRINO, TIPO_BIGLIETTO
Consultazione anagrafica animali	200/giorno	ANIMALE, SPECIE, RECINTO, STORICO_COLLOCAZIONE
Inserimento / modifica animale	5/settimana	ANIMALE, SPECIE, RECINTO, STORICO_COLLOCAZIONE
Inserimento visita medica	10/settimana	VISITA_MEDICA, ANIMALE, DIPENDENTE
Gestione turni	20/settimana	TURNO, DIPENDENTE, AREA
Registrazione ordine giornaliero	5/giorno	FORNITORE, TIPO_CIBO, FORNITORE_CIBO, ORDINE_GIORNALIERO, TRANSAZIONE
Aggiornamento classificazioni	1/mese	STATO_ESISTENZA, HABITAT, FAMIGLIA_SPECIE
Aggiornamento aree/recinti	2/mese	AREA, TIPO_AREA, RECINTO, TIPO_RECINTO
Aggiornamento tipi biglietto	1/mese	TIPO_BIGLIETTO

3.3 Schemi di navigazione e tabelle degli accessi

3.3.1 Schemi di navigazione

- **Vendita biglietti:** selezione del tipo di biglietto, creazione dello scontrino, inserimento del dettaglio e della transazione.
- **Saldo amministrativo:** lettura delle transazioni, aggregazione delle entrate e delle uscite, visualizzazione delle ultime movimentazioni.
- **Statistiche biglietteria:** lettura di scontrini e dettagli, raggruppamento per giorno, settimana o mese.
- **Animali:** ricerca per specie e collocazione attuale nel recinto.
- **Visite mediche:** lettura di animali e veterinari, inserimento o aggiornamento delle visite.
- **Ordini giornalieri:** scelta del fornitore, scelta del tipo di cibo disponibile e registrazione dell'ordine.
- **Gestione struttura:** manutenzione di aree, recinti e classificazioni.

3.3.2 Tabella degli accessi

Operazione	Lecture	Scritture	Accessi principali	Osservazioni
Vendita biglietto	1	3	TIPO_BIGLIETTO, SCONTRINO, DET- TAGLIO_SCONTRINO, TRANSAZIONE	Operazione frequente e transazionale
Saldo amministrativo	2	0	TRANSAZIONE, CATEGO- RIA_TRANSAZIONE	Richiede scansione e aggregazione
Statistiche biglietteria	3	0	SCONTRINO, DET- TAGLIO_SCONTRINO TIPO_BIGLIETTO	Usa raggruppamenti temporali
Consultazione animali	4	0	ANIMALE, SPECIE, RECINTO, STORICO_COLLOCAZIONE	Accesso prevalentemente in lettura
Inserimento animale	4	1	ANIMALE, RECINTO, SPECIE, STORICO_COLLOCAZIONE	Eventuale aggiornamento storico
Inserimento visita medica	3	1	VISITA_MEDICA, ANIMALE, DIPENDENTE	Integrazione con ruolo veterinario

Operazione	Lecture	Scritture	Accessi principali	Osservazioni
Ordine giornaliero	4	2	FORNITORE, TIPO_CIBO, FORNI- TORE_CIBO, OR- DINE_GIORNALIERO, TRANSAZIONE	Scrittura sia contabile sia logistica
Gestione turni	3	1	TURNO, DIPENDENTE, AREA	Usata dal personale amministrativo

3.4 Raffinamento dello schema

Il raffinamento dello schema ha portato alle seguenti scelte:

- tutti gli oggetti principali sono identificati da chiavi surrogate di tipo intero;
- le informazioni ripetitive sono normalizzate in tabelle di dominio (tipo area, tipo recinto, categoria transazione, ecc.);
- le informazioni storiche sono separate dalle entità correnti;
- i legami molti-a-molti sono tradotti tramite tabelle di associazione;
- i dati contabili sono separati dai dati operativi.

3.4.1 Eliminazione di attributi composti e gerarchie

Nel modello relazionale non sono stati mantenuti attributi composti; gli eventuali dettagli composti sono stati scomposti in campi elementari. Inoltre, le gerarchie concettuali sono state assorbite in tabelle separate di supporto, evitando la presenza di specializzazioni esplicite nel modello fisico.

3.4.2 Scelta delle chiavi

La scelta delle chiavi è stata guidata da semplicità e stabilità:

- `id_animale`, `id_specie`, `id_recinto`, `id_area`, `id_dipendente`, `id_utente` e le altre chiavi numeriche sono usate come identificatori primari;
- per alcune tabelle di dominio è possibile imporre un vincolo di unicità sul nome, quando il dominio lo consente;
- le associazioni storiche usano chiavi tecniche che semplificano gli aggiornamenti.

3.5 Analisi delle ridondanze

Nel modello logico sono state individuate alcune informazioni che presentano carattere ridondante o che potrebbero apparire tali a una prima lettura. Per ciascuna si fornisce una giustificazione progettuale e, dove si tratta di un attributo derivabile, un'analisi quantitativa degli accessi secondo il metodo delle tabelle degli accessi: ogni scrittura conta 2, ogni lettura conta 1.

Ridondanza 1 – vivo in ANIMALE

Descrizione

L'attributo **vivo** (booleano) nella tabella **ANIMALE** indica se l'animale è attualmente presente e in vita. Questo valore è *derivabile* dallo storico delle collocazioni: un animale si considera in vita se esiste almeno una riga in **STORICO_COLLOCAZIONE** con **data_fine** NULL. Tuttavia, navigare lo storico per rispondere a questa domanda introduce un costo aggiuntivo che si paga su ogni consultazione dell'anagrafica.

Esiste inoltre una seconda informazione apparentemente sovrapposta: **data_uscita** in **ANIMALE**. Le due informazioni *non sono ridondanti tra loro*: il flag **vivo** descrive lo stato corrente (presente/assente dallo zoo), mentre **data_uscita** conserva la data precisa del trasferimento o del decesso, consentendo di ricostruire la cronologia. Un animale può risultare **vivo = false** senza avere una **data_uscita** valorizzata (dato mancante) e viceversa. La coppia è necessaria per distinguere animale presente, animale trasferito e animale deceduto.

Operazioni pertinenti

1. **Consultazione anagrafica animali** (200/giorno): recupero dell'elenco degli animali con il loro stato corrente.
2. **Aggiornamento dello stato di vita** (5/settimana $\approx$ 1/giorno): quando un animale muore o viene trasferito, il flag va aggiornato e contestualmente va chiusa la riga aperta in **STORICO_COLLOCAZIONE**.

Dati di volume utili

- **ANIMALE**: 150 istanze
- **STORICO_COLLOCAZIONE**: 400 istanze
- Collocazioni medie per animale: $400/150 \approx 2,7$

Tabella degli accessi – con ridondanza

Operazione 1	Costrutto	Accessi	Tipo
ANIMALE	E	1	L
<i>Totale pesato</i>			1

Operazione 2 (aggiornamento)	Costrutto	Accessi	Tipo
ANIMALE	E	1	L
ANIMALE	E	1	S
STORICO_COLLOCAZIONE	R	1	L
STORICO_COLLOCAZIONE	R	1	S
<i>Totale pesato</i>			2L + 2S = 6

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	1	200	200
2	6	1	6
Totale			206

Tabella degli accessi – senza ridondanza

Senza il flag, la consultazione deve verificare l'esistenza di una riga aperta in STORICO_COLLOCAZIONE per ciascun animale.

Operazione 1	Costrutto	Accessi	Tipo
ANIMALE	E	1	L
STORICO_COLLOCAZIONE	R	2,7	L
<i>Totale pesato</i>			3,7

Operazione 2 (aggiornamento)	Costrutto	Accessi	Tipo
STORICO_COLLOCAZIONE	R	1	L
STORICO_COLLOCAZIONE	R	1	S
<i>Totale pesato</i>			3

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	3,7	200	740
2	3	1	3
Totale			743

Decisione

	Con ridondanza	Senza ridondanza
Costo/giorno	206	743

Il costo con ridondanza è circa **3,6 volte inferiore**. La consultazione anagrafica (200/giorno) è l'operazione dominante e beneficia enormemente del flag diretto. **Si mantiene vivo come attributo esplicito.**

Ridondanza 2 – prezzo_pagato_biglietto in DETTAGLIO_SCONTRINO

Descrizione

L'attributo `prezzo_pagato_biglietto` in `DETTAGLIO_SCONTRINO` memorizza il prezzo effettivamente corrisposto al momento dell'acquisto. Tale valore è *nominalmente derivabile* da `TIPO_BIGLIETTO.prezzo`, ma questa derivazione è valida solo se il prezzo non è mai cambiato. I prezzi dei biglietti variano nel tempo: senza questa memorizzazione esplicita sarebbe impossibile ricostruire correttamente i ricavi storici, in quanto il prezzo corrente non rispecchierebbe quello pagato mesi o anni addietro. La ridondanza ha quindi anche una funzione di *storicizzazione*, non solo di efficienza.

Operazioni pertinenti

1. **Vendita di un biglietto** (300/giorno): si legge il prezzo corrente da `TIPO_BIGLIETTO` e lo si copia nel dettaglio scontrino.
2. **Statistiche biglietteria** (20/giorno): calcolo del ricavo per periodo tramite $\text{quantità} \times \text{prezzo_pagato_biglietto}$.
3. **Aggiornamento del prezzo del biglietto** (1/mese $\approx 0,033$ /giorno): modifica di `TIPO_BIGLIETTO.prezzo`; con ridondanza non richiede aggiornamenti ai dettagli storici.

Dati di volume utili

- `TIPO_BIGLIETTO`: 6 istanze
- `DETTAGLIO_SCONTRINO`: 200 000 istanze
- Media righe dettaglio per scontrino: $200\,000/120\,000 \approx 1,7$

Tabella degli accessi – con ridondanza

Operazione 1 (vendita)	Costrutto	Accessi	Tipo
TIPO_BIGLIETTO	E	1	L
SCONTRINO	E	1	S
DETTAGLIO_SCONTRINO	E	1	S
TRANSAZIONE	E	1	S
Totale pesato			1L + 3S = 7

Operazione 2 (statistiche)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	L
DETTAGLIO_SCONTRINO	R	1,7	L
Totale pesato			2,7

Operazione 3 (aggiornamento prezzo)	Costrutto	Accessi	Tipo
TIPO_BIGLIETTO	E	1	L
TIPO_BIGLIETTO	E	1	S
<i>Totale pesato</i>			3

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	7	300	2 100
2	2,7	20	54
3	3	0,033	0,1
Totale			2 154

Tabella degli accessi – senza ridondanza

Senza l'attributo, le statistiche devono raggiungere TIPO_BIGLIETTO per ogni dettaglio, con risultati *potenzialmente incorretti* in caso di variazione del prezzo.

Operazione 1 (vendita)	Costrutto	Accessi	Tipo
TIPO_BIGLIETTO	E	1	L
SCONTRINO	E	1	S
DETTAGLIO_SCONTRINO	E	1	S
TRANSAZIONE	E	1	S
<i>Totale pesato</i>			7

Operazione 2 (statistiche)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	L
DETTAGLIO_SCONTRINO	R	1,7	L
TIPO_BIGLIETTO	E	1,7	L
<i>Totale pesato</i>			4,4

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	7	300	2 100
2	4,4	20	88
3	3	0,033	0,1
Totale			2 188

Decisione

	Con ridondanza	Senza ridondanza
Costo/giorno	2 154	2 188

La differenza di costo è modesta, ma l'eliminazione dell'attributo renderebbe **impossibile ricostruire correttamente i ricavi storici** in presenza di variazioni di prezzo. Si **mantiene** `prezzo_pagato_biglietto`, classificandolo come ridondanza necessaria per la correttezza dei dati storici oltre che per una leggera convenienza di accesso.

Ridondanza 3 – `num_persone` in SCONTRINO

Descrizione

L'attributo `num_persone` nello SCONTRINO indica quante persone sono incluse nell'acquisto. Potrebbe sembrare derivabile sommando le `quantita` dei dettagli: $\text{num_persone} = \sum \text{quantita}$. Tuttavia questa equivalenza non è sempre valida: nel caso di acquisti di gruppo, il numero di persone del gruppo può includere accompagnatori o partecipanti con esenzione non conteggiati come biglietti distinti. Il campo ha quindi un significato semantico autonomo, utile per le statistiche di affluenza e per i controlli operativi.

Operazioni pertinenti

1. **Vendita di un biglietto** (300/giorno): scrittura dello scontrino con il numero di persone dichiarato al momento dell'acquisto.
2. **Statistiche di affluenza** (20/giorno): aggregazione del numero totale di visitatori per periodo.

Dati di volume utili

- SCONTRINO: 120 000 istanze
- DETTAGLIO_SCONTRINO: 200 000 istanze
- Media righe dettaglio per scontrino: $\approx 1,7$

Tabella degli accessi – con ridondanza

Operazione 1 (vendita)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	S
DETTAGLIO_SCONTRINO	E	1	S
<i>Totale pesato</i>			$2S = 4$

Operazione 2 (statistiche)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	L
<i>Totale pesato</i>			1

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	4	300	1 200
2	1	20	20
Totale			1 220

Tabella degli accessi – senza ridondanza

Operazione 1 (vendita)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	S
DETTAGLIO_SCONTRINO	E	1	S
<i>Totale pesato</i>			4

Operazione 2 (statistiche)	Costrutto	Accessi	Tipo
SCONTRINO	E	1	L
DETTAGLIO_SCONTRINO	R	1,7	L
<i>Totale pesato</i>			2,7

Operazione	Costo/occ.	Freq./giorno	Costo/giorno
1	4	300	1 200
2	2,7	20	54
Totale			1 254

Decisione

	Con ridondanza	Senza ridondanza
Costo/giorno	1 220	1 254

La differenza numerica è contenuta (34 accessi/giorno), ma il campo ha un **significato semantico autonomo** non riducibile alla somma dei dettagli. **Si mantiene num_persone come attributo esplicito.**

Considerazioni su STORICO_COLLOCAZIONE e STORICO_STIPENDIO

Le entità STORICO_COLLOCAZIONE e STORICO_STIPENDIO potrebbero apparire ridondanti rispetto a una semplice FK diretta sull'animale o sul dipendente. In realtà non si tratta di ridondanza ma di una precisa *scelta di modellazione storica*: l'obiettivo non è memorizzare il dato corrente, bensì conservare l'intera sequenza delle variazioni nel tempo.

- **STORICO_COLLOCAZIONE**: memorizza in quale recinto si trovava ciascun animale e per quale periodo. Senza questa entità sarebbe impossibile sapere dove si trovava un animale in una data passata, quante volte è stato spostato o per quanto tempo ha occupato un determinato recinto. Una FK diretta da **ANIMALE** a **RECINTO** fornirebbe solo la collocazione attuale, perdendo tutta la storia degli spostamenti.
- **STORICO_STIPENDIO**: conserva l'evoluzione della retribuzione oraria di ogni dipendente nel tempo. Senza questa entità sarebbe impossibile calcolare il costo del lavoro per periodi passati o ricostruire la storia contrattuale di un dipendente. Una semplice FK o un campo **prezzo_orario** direttamente in **DIPENDENTE** rifletterebbe solo il valore attuale.

In entrambi i casi la struttura adottata è quella del *tipo intervallo*: ogni riga registra un periodo di validità (**data_inizio**, **data_fine**) e la riga con **data_fine** NULL rappresenta la situazione corrente. Non si tratta quindi di duplicazione, bensì di un **requisito esplicito di storicizzazione** emerso dall'intervista con il committente.

Considerazioni su tipo e id_categoria in TRANSAZIONE

In **TRANSAZIONE** coesistono due informazioni apparentemente simili: **tipo** (valori ENTRATA/USCITA) e il riferimento a **CATEGORIA_TRANSAZIONE**. Non si tratta di ridondanza:

- **tipo** è il *segno contabile* della transazione: determina se il movimento incrementa o decrementa il saldo. Viene usato direttamente nelle aggregazioni per il calcolo del saldo corrente.
- **id_categoria** è la *classificazione gestionale*: descrive la natura economica della spesa o dell'entrata (es. alimentazione, manutenzione, biglietteria, stipendi). Viene usato per le analisi di dettaglio e per il budget.

Sebbene una categoria possa implicitamente determinare il segno (es. tutte le vendite biglietti sono entrate), esistono categorie che possono generare movimenti in entrambe le direzioni (es. rimborsi, rettifiche). Mantenere **tipo** come campo autonomo garantisce la correttezza del saldo anche in questi casi limite e rende le query di aggregazione più semplici ed efficienti, evitando un join aggiuntivo con **CATEGORIA_TRANSAZIONE** per determinare il segno.

Riepilogo delle scelte

Elemento	Con rid.	Senza rid.	Decisione
animale.vivo	206	743	Mantenuto (efficienza)
dettaglio_sco.prezzo_pagato	2 154	2 188	Mantenuto (correttezza storica)
scontrino.num_person	1 220	1 254	Mantenuto (semantica)

Elemento	Con rid.	Senza rid.	Decisione
STORICO_COLLOCAZIONE	—	—	Non è ridondanza (storico)
STORICO_STIPENDIO	—	—	Non è ridondanza (storico)
tipo in TRANSAZIONE	—	—	Non è ridondanza (semantica)

3.6 Traduzione di entità e associazioni in relazioni

Lo schema relazionale finale è composto dalle seguenti relazioni.

Relazione	Contenuto sintetico
STATO_ESISTENZA	dati di classificazione dello stato di conservazione delle specie
HABITAT	ambienti naturali di riferimento
FAMIGLIA_SPECIE	famiglie zoologiche
TIPO_AREA	tipologia delle aree dello zoo
TIPO_RECINTO	tipologia dei recinti
MANSIONE	ruoli del personale
TIPO_CIBO	categorie di cibo per gli animali
TIPO_FORNITURA	categorie di fornitura
TIPO_BIGLIETTO	tariffe dei biglietti
CATEGORIA_TRANSAZ SPECIE	classificazione delle transazioni economiche anagrafica delle specie con riferimenti a stato, habitat e famiglia
DIETA	fabbisogni alimentari delle specie
AREA	aree dello zoo con tipologia associata
RECINTO	recinti con area e tipologia
ANIMALE	anagrafica degli animali
STORICO_COLLOCAZIO DIPENDENTE	permanenza degli animali nei recinti nel tempo anagrafica del personale
STORICO_STIPENDIO	variazioni della retribuzione nel tempo
TURNO	turni di lavoro del personale
VISITA_MEDICA	visite veterinarie degli animali
UTENTE	account del gestionale e ruolo applicativo
FORNITORE	anagrafica fornitori
FORNITORE_CIBO	listino cibi per fornitore
ORDINE_GIORNALIERO	ordini di cibo effettuati
SCONTRINO	testata di vendita della biglietteria
DETTAGLIO_SCONTRINO	dettaglio dei biglietti venduti
TRANSAZIONE	movimenti economici di entrata e uscita

3.7 Schema relazionale finale

Il progetto finale, come implementato nel database PostgreSQL, adotta il seguente insieme di tabelle principali:

- classificazione: stato_esistenza, habitat, famiglia_specie, specie, dieta;
- spazi: tipo_area, tipo_recinto, area, recinto, storico_collocazione;
- personale: mansione, dipendente, storico_stipendio, turno, utente;
- sanità: visita_medica;
- contabilità: categoria_transazione, transazione, scontrino, dettaglio_scontrino, tipo_biglietto;
- approvvigionamenti: tipo_cibo, tipo_fornitura, fornitore, fornitore_cibo, ordine_giornaliero;
- altre funzionalità:

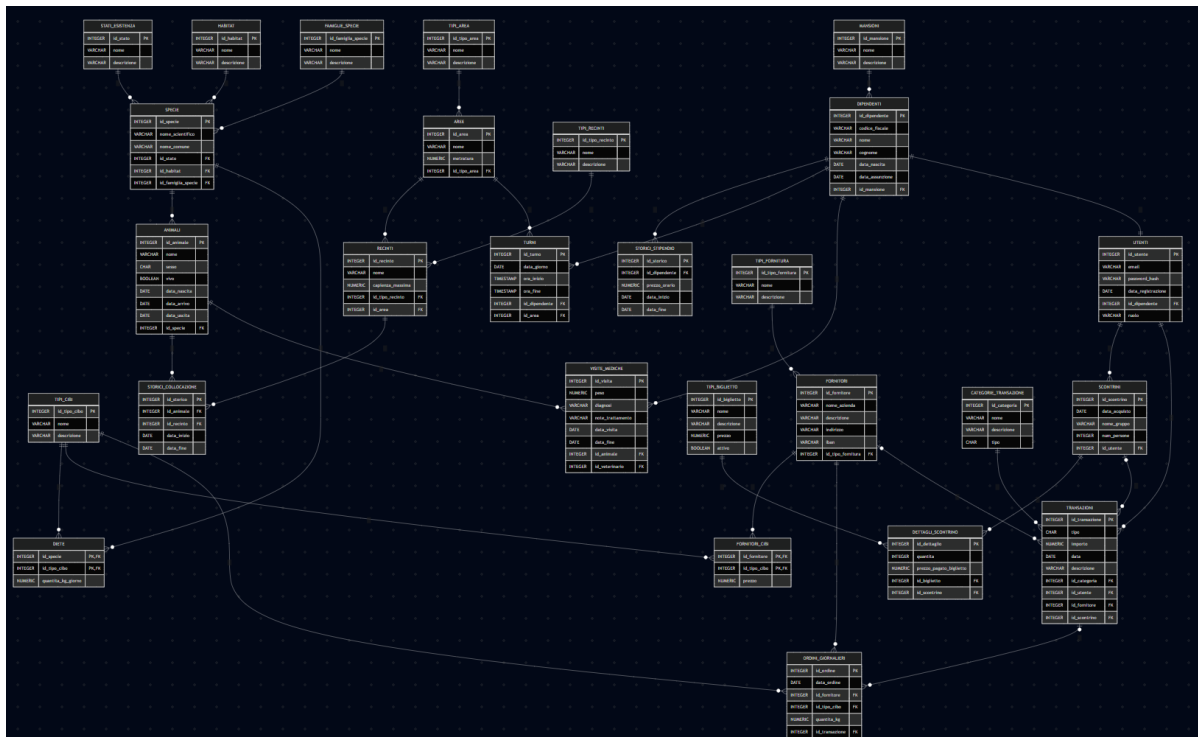


Figure 3.1: Schema relazionale finale

Link allo schema relazionale: [link mermeid live editor](#)

3.8 Traduzione delle operazioni in query SQL

SELECT – Lettura dei dati

3.8.1 Lettura

la lettura di tutti i record (findAll). Per ogni tabella del sistema è disponibile una query generica di lettura completa, generata dinamicamente dalla classe astratta AbstractCrudDao. **la traduzione è quella sotto al codice sql. Non è quella sopra al codice**

Animale

```
SELECT * FROM animale
```

traduzione Seleziona tutti gli animali presenti nel database.

```
SELECT * FROM animale WHERE id_animale = ?
```

traduzione Seleziona un singolo animale identificato dal suo ID.

```
SELECT * FROM animale WHERE id_specie = ?
```

traduzione Seleziona tutti gli animali appartenenti a una determinata specie.

```
SELECT * FROM animale WHERE vivo = true
```

traduzione Seleziona tutti gli animali attualmente in vita.

```
SELECT * FROM animale WHERE vivo = false
```

traduzione Seleziona tutti gli animali deceduti o usciti dallo zoo.

Area

```
SELECT * FROM area
```

traduzione Seleziona tutte le aree dello zoo.

```
SELECT * FROM area WHERE id_area = ?
```

traduzione Seleziona una singola area identificata dal suo ID.

```
SELECT * FROM area WHERE id_tipo_area = ?
```

traduzione Seleziona tutte le aree di un determinato tipo (es. savana, acquatica, ecc.).

Categoria Transazione

```
SELECT * FROM categoria_transazione
```

traduzione Seleziona tutte le categorie di transazione disponibili.

```
SELECT * FROM categoria_transazione WHERE id_categoria = ?
```

traduzione Seleziona una singola categoria di transazione tramite ID.

```
SELECT * FROM categoria_transazione WHERE nome = ?
```

traduzione Seleziona una categoria di transazione cercando per nome.

```
SELECT * FROM categoria_transazione WHERE tipo = ?
```

traduzione Seleziona tutte le categorie di transazione di un certo tipo (es. entrata, uscita).

Dettaglio Scontrino

```
SELECT * FROM dettaglio_scontrino
```

traduzione Seleziona tutti i dettagli degli scontrini emessi.

```
SELECT * FROM dettaglio_scontrino WHERE id_dettaglio = ?
```

traduzione Seleziona un singolo dettaglio di scontrino tramite ID.

```
SELECT * FROM dettaglio_scontrino WHERE id_scontrino = ?
```

traduzione Seleziona tutte le righe di dettaglio associate a uno specifico scontrino.

```
SELECT * FROM dettaglio_scontrino WHERE id_biglietto = ?
```

traduzione Seleziona tutti i dettagli scontrino che riguardano un determinato tipo di biglietto.

Dieta

```
SELECT * FROM dieta
```

traduzione Seleziona tutte le diete definite nel sistema.

```
SELECT * FROM dieta WHERE id_specie = ? AND id_tipo_cibo = ?
```

traduzione Seleziona la dieta di una specifica specie per uno specifico tipo di cibo (chiave composta).

```
SELECT * FROM dieta WHERE id_specie = ?
```

traduzione Seleziona tutte le voci di dieta associate a una determinata specie animale.

```
SELECT * FROM dieta WHERE id_tipo_cibo = ?
```

traduzione Seleziona tutte le diete che prevedono un determinato tipo di cibo.

Dipendente

```
SELECT * FROM dipendente
```

traduzione Seleziona tutti i dipendenti dello zoo.

```
SELECT * FROM dipendente WHERE id_dipendente = ?
```

traduzione Seleziona un singolo dipendente tramite il suo ID.

```
SELECT * FROM dipendente WHERE codice_fiscale = ?
```

traduzione Seleziona un dipendente cercando per codice fiscale.

```
SELECT * FROM dipendente WHERE id_mansione = ?
```

traduzione Seleziona tutti i dipendenti che ricoprono una determinata mansione.

Famiglia Specie

```
SELECT * FROM famiglia_specie
```

traduzione Seleziona tutte le famiglie tassonomiche presenti nel sistema.

```
SELECT * FROM famiglia_specie WHERE id_famiglia_specie = ?
```

traduzione Seleziona una singola famiglia tassonomica tramite ID.

```
SELECT * FROM famiglia_specie WHERE nome = ?
```

traduzione Seleziona una famiglia tassonomica cercando per nome.

Fornitore

```
SELECT * FROM fornitore
```

traduzione Seleziona tutti i fornitori registrati nel sistema.

```
SELECT * FROM fornitore WHERE id_fornitore = ?
```

traduzione Seleziona un singolo fornitore tramite ID.

```
SELECT * FROM fornitore WHERE id_tipo_fornitura = ?
```

traduzione Seleziona tutti i fornitori che offrono un determinato tipo di fornitura.

Fornitore Cibo

```
SELECT * FROM fornitore_cibo
```

traduzione Seleziona tutte le associazioni tra fornitori e tipi di cibo (con i relativi prezzi).

```
SELECT * FROM fornitore_cibo WHERE id_fornitore = ? AND id_tipo_cibo = ?
```

traduzione Seleziona il prezzo di un tipo di cibo specifico offerto da uno specifico fornitore (chiave composta).

```
SELECT * FROM fornitore_cibo WHERE id_fornitore = ?
```

traduzione Seleziona tutti i cibi forniti da uno specifico fornitore con i relativi prezzi.

```
SELECT * FROM fornitore_cibo WHERE id_tipo_cibo = ?
```

traduzione Seleziona tutti i fornitori che vendono un determinato tipo di cibo.

Habitat

```
SELECT * FROM habitat
```

traduzione Seleziona tutti gli habitat definiti nel sistema.

```
SELECT * FROM habitat WHERE id_habitat = ?
```

traduzione Seleziona un singolo habitat tramite ID.

```
SELECT * FROM habitat WHERE nome = ?
```

traduzione Seleziona un habitat cercando per nome.

Mansione

```
SELECT * FROM mansione
```

traduzione Seleziona tutte le mansioni disponibili per i dipendenti.

```
SELECT * FROM mansione WHERE id_mansione = ?
```

traduzione Seleziona una singola mansione tramite ID.

```
SELECT * FROM mansione WHERE nome = ?
```

traduzione Seleziona una mansione cercando per nome.

Ordine Giornaliero

```
SELECT * FROM ordine_giornaliero
```

traduzione Seleziona tutti gli ordini giornalieri di cibo.

```
SELECT * FROM ordine_giornaliero WHERE id_ordine = ?
```

traduzione Seleziona un singolo ordine giornaliero tramite ID.

```
SELECT * FROM ordine_giornaliero WHERE data_ordine = ? ORDER BY id_ordine DESC
```

traduzione Seleziona tutti gli ordini effettuati in una determinata data, ordinati dal più recente.

```
SELECT * FROM ordine_giornaliero WHERE id_fornitore = ? ORDER BY data_ordine  
DESC
```

traduzione Seleziona tutti gli ordini effettuati a uno specifico fornitore, dal più recente.

```
SELECT * FROM ordine_giornaliero WHERE id_tipo_cibo = ? ORDER BY data_ordine  
DESC
```

traduzione Seleziona tutti gli ordini relativi a uno specifico tipo di cibo, dal più recente.

```
SELECT * FROM ordine_giornaliero WHERE id_transazione = ? ORDER BY data_ordine  
DESC
```

traduzione Seleziona tutti gli ordini associati a una determinata transazione finanziaria, dal più recente.

Recinto

```
SELECT * FROM recinto
```

traduzione Seleziona tutti i recinti dello zoo.

```
SELECT * FROM recinto WHERE id_recinto = ?
```

traduzione Seleziona un singolo recinto tramite ID.

```
SELECT * FROM recinto WHERE id_area = ?
```

traduzione Seleziona tutti i recinti appartenenti a una determinata area.

```
SELECT * FROM recinto WHERE id_tipo_recinto = ?
```

traduzione Seleziona tutti i recinti di un determinato tipo.

```
SELECT * FROM recinto WHERE capienza_massima >= ?
```

traduzione Seleziona tutti i recinti con una capienza massima pari o superiore al valore indicato.

Scontrino

```
SELECT * FROM scontrino
```

traduzione Seleziona tutti gli scontrini emessi.

```
SELECT * FROM scontrino WHERE id_scontrino = ?
```

traduzione Seleziona un singolo scontrino tramite ID.

```
SELECT * FROM scontrino WHERE data_acquisto = ? ORDER BY id_scontrino DESC
```

traduzione Seleziona tutti gli scontrini emessi in una determinata data, dal più recente.

```
SELECT * FROM scontrino WHERE id_utente = ? ORDER BY data_acquisto DESC
```

traduzione Seleziona tutti gli scontrini associati a uno specifico utente, dal più recente per data d'acquisto.

Specie

```
SELECT * FROM specie
```

traduzione Seleziona tutte le specie animali registrate.

```
SELECT * FROM specie WHERE id_specie = ?
```

traduzione Seleziona una singola specie tramite ID.

```
SELECT * FROM specie WHERE id_habitat = ?
```

traduzione Seleziona tutte le specie che vivono in un determinato habitat.

```
SELECT * FROM specie WHERE id_stato = ?
```

traduzione Seleziona tutte le specie con un determinato stato di esistenza (es. in via di estinzione).

```
SELECT * FROM specie WHERE id_famiglia_specie = ?
```

traduzione Seleziona tutte le specie appartenenti a una determinata famiglia tassonomica.

Stato Esistenza

```
SELECT * FROM stato_esistenza
```

traduzione Seleziona tutti gli stati di esistenza (es. vulnerabile, endangered, estinto, ecc.).

```
SELECT * FROM stato_esistenza WHERE id_stato = ?
```

traduzione Seleziona un singolo stato di esistenza tramite ID.

```
SELECT * FROM stato_esistenza WHERE nome = ?
```

traduzione Seleziona uno stato di esistenza cercando per nome.

Storico Collocazione

```
SELECT * FROM storico_collocazione
```

traduzione Seleziona l'intero storico delle collocazioni degli animali nei recinti.

```
SELECT * FROM storico_collocazione WHERE id_storico = ?
```

traduzione Seleziona un singolo record dello storico tramite ID.

```
SELECT * FROM storico_collocazione WHERE id_animale = ? ORDER BY data_inizio  
DESC
```

traduzione Seleziona tutto lo storico di collocazione di uno specifico animale, dal più recente.

```
SELECT * FROM storico_collocazione WHERE id_recinto = ? ORDER BY data_inizio  
DESC
```

traduzione Seleziona tutto lo storico di occupazione di uno specifico recinto, dal più recente.

Storico Stipendio

```
SELECT * FROM storico_stipendio
```

traduzione Seleziona l'intero storico degli stipendi dei dipendenti.

```
SELECT * FROM storico_stipendio WHERE id_storico = ?
```

traduzione Seleziona un singolo record dello storico stipendi tramite ID.

```
SELECT * FROM storico_stipendio WHERE id_dipendente = ? ORDER BY data_inizio  
DESC
```

traduzione Seleziona tutto lo storico stipendi di uno specifico dipendente, dal più recente.

Tipo Area

```
SELECT * FROM tipo_area
```

traduzione Seleziona tutti i tipi di area disponibili.

```
SELECT * FROM tipo_area WHERE id_tipo_area = ?
```

traduzione Seleziona un singolo tipo di area tramite ID.

```
SELECT * FROM tipo_area WHERE nome = ?
```

traduzione Seleziona un tipo di area cercando per nome.

Tipo Biglietto

```
SELECT * FROM tipo_biglietto
```

traduzione Seleziona tutti i tipi di biglietto definiti nel sistema.

```
SELECT * FROM tipo_biglietto WHERE id_biglietto = ?
```

traduzione Seleziona un singolo tipo di biglietto tramite ID.

```
SELECT * FROM tipo_biglietto WHERE nome = ?
```

traduzione Seleziona un tipo di biglietto cercando per nome.

```
SELECT * FROM tipo_biglietto WHERE attivo = ?
```

traduzione Seleziona tutti i tipi di biglietto attivi (o non attivi) in base al flag booleano.

Tipo Cibo

```
SELECT * FROM tipo_cibo
```

traduzione Seleziona tutti i tipi di cibo registrati.

```
SELECT * FROM tipo_cibo WHERE id_tipo_cibo = ?
```

traduzione Seleziona un singolo tipo di cibo tramite ID.

```
SELECT * FROM tipo_cibo WHERE nome = ?
```

traduzione Seleziona un tipo di cibo cercando per nome.

Tipo Fornitura

```
SELECT * FROM tipo_fornitura
```

traduzione Seleziona tutti i tipi di fornitura disponibili.

```
SELECT * FROM tipo_fornitura WHERE id_tipo_fornitura = ?
```

traduzione Seleziona un singolo tipo di fornitura tramite ID.

```
SELECT * FROM tipo_fornitura WHERE nome = ?
```

traduzione Seleziona un tipo di fornitura cercando per nome.

Tipo Recinto

```
SELECT * FROM tipo_recinto
```

traduzione Seleziona tutti i tipi di recinto presenti.

```
SELECT * FROM tipo_recinto WHERE id_tipo_recinto = ?
```

traduzione Seleziona un singolo tipo di recinto tramite ID.

```
SELECT * FROM tipo_recinto WHERE nome = ?
```

traduzione Seleziona un tipo di recinto cercando per nome.

Transazione

```
SELECT * FROM transazione
```

traduzione Seleziona tutte le transazioni finanziarie registrate.

```
SELECT * FROM transazione WHERE id_transazione = ?
```

traduzione Seleziona una singola transazione tramite ID.

```
SELECT * FROM transazione WHERE tipo = ? ORDER BY data DESC, id_transazione  
DESC
```

traduzione Seleziona tutte le transazioni di un certo tipo, ordinate per data decrescente.

```
SELECT * FROM transazione WHERE data = ? ORDER BY id_transazione DESC
```

traduzione Seleziona tutte le transazioni effettuate in una determinata data, dalla più recente.

```
SELECT * FROM transazione WHERE data >= ? AND data <= ? ORDER BY data DESC,  
id_transazione DESC
```

traduzione Seleziona tutte le transazioni comprese in un intervallo di date, ordinate per data decrescente.

```
SELECT * FROM transazione WHERE id_categoria = ? ORDER BY data DESC,  
id_transazione DESC
```

traduzione Seleziona tutte le transazioni appartenenti a una determinata categoria, ordinate per data decrescente.

```
SELECT * FROM transazione WHERE id_utente = ? ORDER BY data DESC,  
id_transazione DESC
```

traduzione Seleziona tutte le transazioni associate a uno specifico utente, ordinate per data decrescente.


```
SELECT * FROM transazione WHERE id_fornitore = ? ORDER BY data DESC,  
id_transazione DESC
```

traduzione Seleziona tutte le transazioni relative a uno specifico fornitore, ordinate per data decrescente.

```
SELECT * FROM transazione WHERE id_scontrino = ? ORDER BY data DESC,  
id_transazione DESC
```

traduzione Seleziona tutte le transazioni collegate a uno specifico scontrino, ordinate per data decrescente.

Turno

```
SELECT * FROM turno
```

traduzione Seleziona tutti i turni lavorativi registrati.

```
SELECT * FROM turno WHERE id_turno = ?
```

traduzione Seleziona un singolo turno tramite ID.

```
SELECT * FROM turno WHERE data_giorno = ? ORDER BY ora_inizio
```

traduzione Seleziona tutti i turni di un determinato giorno, ordinati per ora di inizio.

```
SELECT * FROM turno WHERE data_giorno >= ? AND data_giorno <= ? ORDER BY  
data_giorno, ora_inizio
```

traduzione Seleziona tutti i turni compresi in un intervallo di date, ordinati per giorno e ora di inizio.

```
SELECT * FROM turno WHERE id_dipendente = ? ORDER BY ora_inizio
```

traduzione Seleziona tutti i turni assegnati a uno specifico dipendente, ordinati per ora di inizio.

```
SELECT * FROM turno WHERE id_area = ? ORDER BY ora_inizio
```

traduzione Seleziona tutti i turni relativi a una determinata area, ordinati per ora di inizio.

Utente

```
SELECT * FROM utente
```

traduzione Seleziona tutti gli utenti registrati nel sistema.

```
SELECT * FROM utente WHERE id_utente = ?
```

traduzione Seleziona un singolo utente tramite ID.

```
SELECT * FROM utente WHERE email = ?
```

traduzione Seleziona un utente cercando per indirizzo email (usato tipicamente per il login).

```
SELECT * FROM utente WHERE ruolo = ?
```

traduzione Seleziona tutti gli utenti che hanno un determinato ruolo nel sistema.

```
SELECT * FROM utente WHERE id_dipendente = ?
```

traduzione Seleziona l'utente associato a uno specifico dipendente.

Visita Medica

```
SELECT * FROM visita_medica
```

traduzione Seleziona tutte le visite mediche registrate.

```
SELECT * FROM visita_medica WHERE id_visita = ?
```

traduzione Seleziona una singola visita medica tramite ID.

```
SELECT * FROM visita_medica WHERE id_animale = ? ORDER BY data_visita DESC
```

traduzione Seleziona tutte le visite mediche di uno specifico animale, dalla più recente.

```
SELECT * FROM visita_medica WHERE data_visita = ? ORDER BY id_visita DESC
```

traduzione Seleziona tutte le visite mediche effettuate in una determinata data, dalla più recente.

```
SELECT * FROM visita_medica WHERE id_veterinario = ? ORDER BY data_visita DESC
```

traduzione Seleziona tutte le visite mediche eseguite da uno specifico veterinario, dalla più recente.

3.8.2 Inserimento

INSERT – Inserimento dei dati

```
INSERT INTO animale (nome, sesso, vivo, data_nascita, data_arrivo, data_uscita,  
id_specie)  
VALUES (?, ?, ?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo animale con nome, sesso, stato in vita, date di nascita, arrivo e uscita, e la specie di appartenenza.

```
INSERT INTO area (nome, metratura, id_tipo_area) VALUES (?, ?, ?)
```

traduzione Inserisce una nuova area dello zoo specificando il nome, la superficie in metri quadri e il tipo di area.

```
INSERT INTO categoria_transazione (nome, descrizione, tipo) VALUES (?, ?, ?)
```

traduzione Inserisce una nuova categoria di transazione con nome, descrizione e tipo (entrata/uscita).

```
INSERT INTO dettaglio_scontrino (id_scontrino, id_biglietto, quantita,  
prezzo_pagato_biglietto)  
VALUES (?, ?, ?, ?)
```

traduzione Inserisce una riga di dettaglio in uno scontrino, specificando il biglietto, la quantità acquistata e il prezzo pagato.

```
INSERT INTO dieta (id_specie, id_tipo_cibo, quantita_kg_giorno) VALUES (?, ?,  
?)
```

traduzione Inserisce una voce di dieta indicando quanti kg al giorno di un determinato cibo una certa specie deve ricevere.

```
INSERT INTO dipendente (codice_fiscale, nome, cognome, data_nascita,  
data_assunzione, id_mansione)  
VALUES (?, ?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo dipendente con i dati anagrafici, la data di assunzione e la mansione assegnata.

```
INSERT INTO famiglia_specie (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce una nuova famiglia tassonomica con nome e descrizione.

```
INSERT INTO fornitore (nome_azienda, descrizione, indirizzo, iban,  
id_tipo_fornitura)  
VALUES (?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo fornitore con ragione sociale, descrizione, indirizzo, coordinate bancarie e tipo di fornitura offerta.

```
INSERT INTO fornitore_cibo (id_fornitore, id_tipo_cibo, prezzo) VALUES (?, ?,  
?)
```

traduzione Inserisce una nuova associazione fornitore-cibo, indicando il prezzo al kg praticato da quel fornitore per quel cibo.

```
INSERT INTO habitat (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo tipo di habitat con nome e descrizione.

```
INSERT INTO mansione (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce una nuova mansione lavorativa con nome e descrizione.

```
INSERT INTO ordine_giornaliero (data_ordine, quantita_kg, id_fornitore,  
id_tipo_cibo, id_transazione)  
VALUES (?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo ordine giornaliero di cibo specificando la data, la quantità in kg, il fornitore, il tipo di cibo e la transazione associata.

```
INSERT INTO recinto (nome, capienza_massima, id_area, id_tipo_recinto) VALUES  
(?, ?, ?, ?)
```

traduzione Inserisce un nuovo recinto con nome, capienza massima, area di appartenenza e tipo di recinto.

```
INSERT INTO scontrino (data_acquisto, nome_gruppo, num_persone, id_utente)  
VALUES (?, ?, ?, ?)
```

traduzione Inserisce un nuovo scontrino con la data di acquisto, il nome del gruppo, il numero di persone e l'utente che ha effettuato l'acquisto.

```
INSERT INTO specie (nome_scientifico, nome_comune, id_habitat, id_stato,  
id_famiglia_specie)  
VALUES (?, ?, ?, ?, ?)
```

traduzione Inserisce una nuova specie animale con nome scientifico, nome comune, habitat, stato di esistenza e famiglia tassonomica.

```
INSERT INTO stato_esistenza (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo stato di esistenza (es. LC, VU, EN, CR, EX) con nome e descrizione.

```
INSERT INTO storico_collocazione (id_animale, id_recinto, data_inizio,  
data_fine) VALUES (?, ?, ?, ?)
```

traduzione Inserisce una nuova voce nello storico collocazioni, indicando in quale recinto si trovava un animale e per quale periodo.

```
INSERT INTO storico_stipendio (id_dipendente, prezzo_orario, data_inizio,  
data_fine) VALUES (?, ?, ?, ?)
```

traduzione Inserisce una nuova voce nello storico stipendi di un dipendente con il prezzo orario e il periodo di validità.

```
INSERT INTO tipo_area (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo tipo di area con nome e descrizione.

```
INSERT INTO tipo_biglietto (nome, descrizione, prezzo, attivo) VALUES (?, ?, ?,  
?)
```

traduzione Inserisce un nuovo tipo di biglietto con nome, descrizione, prezzo e flag di attivazione.

```
INSERT INTO tipo_cibo (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo tipo di cibo con nome e descrizione.

```
INSERT INTO tipo_fornitura (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo tipo di fornitura con nome e descrizione.

```
INSERT INTO tipo_recinto (nome, descrizione) VALUES (?, ?)
```

traduzione Inserisce un nuovo tipo di recinto con nome e descrizione.

```
INSERT INTO transazione (tipo, importo, data, descrizione, id_categoria,  
    id_utente, id_fornitore, id_scontrino)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
```

traduzione Inserisce una nuova transazione finanziaria specificando tipo, importo, data, descrizione, categoria, utente, fornitore e scontrino associati.

```
INSERT INTO turno (data_giorno, ora_inizio, ora_fine, id_dipendente, id_area)  
VALUES (?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo turno lavorativo con il giorno, l'orario di inizio e fine, il dipendente assegnato e l'area di competenza.

```
INSERT INTO utente (email, password_hash, data_registrazione, ruolo,  
    id_dipendente)  
VALUES (?, ?, ?, ?, ?)
```

traduzione Inserisce un nuovo utente con email, hash della password, data di registrazione, ruolo e il dipendente associato (se presente).

```
INSERT INTO visita_medica (peso, diagnosi, note_trattamento, data_visita,  
    data_fine, id_animale, id_veterinario)  
VALUES (?, ?, ?, ?, ?, ?, ?)
```

traduzione Inserisce una nuova visita medica con peso rilevato, diagnosi, note di trattamento, date di inizio e fine, l'animale visitato e il veterinario responsabile.

3.8.3 Update

UPDATE – Aggiornamento dei dati

```
UPDATE animale
SET nome = ?, sesso = ?, vivo = ?, data_nascita = ?, data_arrivo = ?,
    data_uscita = ?, id_specie = ?
WHERE id_animale = ?
```

traduzione Aggiorna tutti i dati di un animale esistente identificato dal suo ID.

```
UPDATE area SET nome = ?, metratura = ?, id_tipo_area = ? WHERE id_area = ?
```

traduzione Aggiorna nome, metratura e tipo di una area esistente.

```
UPDATE categoria_transazione SET nome = ?, descrizione = ?, tipo = ? WHERE
id_categoria = ?
```

traduzione Aggiorna nome, descrizione e tipo di una categoria di transazione esistente.

```
UPDATE dettaglio_scontrino
SET id_scontrino = ?, id_biglietto = ?, quantita = ?, prezzo_pagato_biglietto =
?
WHERE id_dettaglio = ?
```

traduzione Aggiorna i dati di un dettaglio scontrino esistente.

```
UPDATE dieta SET id_specie = ?, id_tipo_cibo = ?, quantita_kg_giorno = ?
WHERE id_specie = ? AND id_tipo_cibo = ?
```

traduzione Aggiorna la quantità giornaliera di un cibo per una determinata specie (chiave composta).

```
UPDATE dipendente
SET codice_fiscale = ?, nome = ?, cognome = ?, data_nascita = ?,
    data_assunzione = ?, id_mansione = ?
WHERE id_dipendente = ?
```

traduzione Aggiorna tutti i dati anagrafici e la mansione di un dipendente.

```
UPDATE famiglia_specie SET nome = ?, descrizione = ? WHERE id_famiglia_specie =
?
```

traduzione Aggiorna nome e descrizione di una famiglia tassonomica.

```
UPDATE fornitore
SET nome_azienda = ?, descrizione = ?, indirizzo = ?, iban = ?,
    id_tipo_fornitura = ?
WHERE id_fornitore = ?
```

traduzione Aggiorna tutti i dati di un fornitore esistente.

```
UPDATE fornitore_cibo SET prezzo = ? WHERE id_fornitore = ? AND id_tipo_cibo =
?
```

traduzione Aggiorna il prezzo praticato da un fornitore per un determinato tipo di cibo (chiave composta).

```
UPDATE habitat SET nome = ?, descrizione = ? WHERE id_habitat = ?
```

traduzione Aggiorna nome e descrizione di un habitat.

```
UPDATE mansione SET nome = ?, descrizione = ? WHERE id_mansione = ?
```

traduzione Aggiorna nome e descrizione di una mansione.

```
UPDATE ordine_giornaliero  
SET data_ordine = ?, quantita_kg = ?, id_fornitore = ?, id_tipo_cibo = ?,  
    id_transazione = ?  
WHERE id_ordine = ?
```

traduzione Aggiorna tutti i dati di un ordine giornaliero esistente.

```
UPDATE recinto SET nome = ?, capienza_massima = ?, id_area = ?, id_tipo_recinto  
    = ?  
WHERE id_recinto = ?
```

traduzione Aggiorna nome, capienza, area e tipo di un recinto.

```
UPDATE scontrino SET data_acquisto = ?, nome_gruppo = ?, num_persone = ?,  
    id_utente = ?  
WHERE id_scontrino = ?
```

traduzione Aggiorna i dati di uno scontrino esistente.

```
UPDATE specie  
SET nome_scientifico = ?, nome_comune = ?, id_habitat = ?, id_stato = ?,  
    id_famiglia_specie = ?  
WHERE id_specie = ?
```

traduzione Aggiorna tutti i dati di una specie animale.

```
UPDATE stato_esistenza SET nome = ?, descrizione = ? WHERE id_stato = ?
```

traduzione Aggiorna nome e descrizione di uno stato di esistenza.

```
UPDATE storico_collocazione  
SET id_animale = ?, id_recinto = ?, data_inizio = ?, data_fine = ?  
WHERE id_storico = ?
```

traduzione Aggiorna i dati di una voce dello storico collocazioni (es. per impostare la data di fine).

```
UPDATE storico_stipendio  
SET id_dipendente = ?, prezzo_orario = ?, data_inizio = ?, data_fine = ?  
WHERE id_storico = ?
```

traduzione Aggiorna i dati di una voce dello storico stipendi (es. per chiudere un periodo).

```
UPDATE tipo_area SET nome = ?, descrizione = ? WHERE id_tipo_area = ?
```

traduzione Aggiorna nome e descrizione di un tipo di area.

```
UPDATE tipo_biglietto SET nome = ?, descrizione = ?, prezzo = ?, attivo = ?  
    WHERE id_biglietto = ?
```

traduzione Aggiorna tutti i dati di un tipo di biglietto, incluso il prezzo e il flag di attivazione.

```
UPDATE tipo_cibo SET nome = ?, descrizione = ? WHERE id_tipo_cibo = ?
```

traduzione Aggiorna nome e descrizione di un tipo di cibo.

```
UPDATE tipo_fornitura SET nome = ?, descrizione = ? WHERE id_tipo_fornitura = ?
```

traduzione Aggiorna nome e descrizione di un tipo di fornitura.

```
UPDATE tipo_recinto SET nome = ?, descrizione = ? WHERE id_tipo_recinto = ?
```

traduzione Aggiorna nome e descrizione di un tipo di recinto.

```
UPDATE transazione  
SET tipo = ?, importo = ?, data = ?, descrizione = ?, id_categoria = ?,  
    id_utente = ?, id_fornitore = ?, id_scontrino = ?  
WHERE id_transazione = ?
```

traduzione Aggiorna tutti i dati di una transazione finanziaria esistente.

```
UPDATE turno SET data_giorno = ?, ora_inizio = ?, ora_fine = ?, id_dipendente =  
    ?, id_area = ?  
WHERE id_turno = ?
```

traduzione Aggiorna i dati di un turno lavorativo esistente.

```
UPDATE utente SET email = ?, password_hash = ?, data_registrazione = ?, ruolo =  
    ?, id_dipendente = ?  
WHERE id_utente = ?
```

traduzione Aggiorna tutti i dati di un utente, incluse email, password e ruolo.

```
UPDATE visita_medica  
SET peso = ?, diagnosi = ?, note_trattamento = ?, data_visita = ?, data_fine =  
    ?, id_animale = ?, id_veterinario = ?  
WHERE id_visita = ?
```

traduzione Aggiorna tutti i dati di una visita medica esistente.

3.8.4 Cancellazione

DELETE – Cancellazione dei dati

Le query di cancellazione sono generate dinamicamente dalla classe astratta `AbstractCrudDao` e `AbstractCompositeCrudDao`. Di seguito le forme concrete per ogni tabella.

```
DELETE FROM animale WHERE id_animale = ?
```

traduzione Elimina un animale dal database tramite il suo ID.

```
DELETE FROM area WHERE id_area = ?
```

traduzione Elimina un'area tramite il suo ID.

```
DELETE FROM categoria_transazione WHERE id_categoria = ?
```

traduzione Elimina una categoria di transazione tramite il suo ID.

```
DELETE FROM dettaglio_scontrino WHERE id_dettaglio = ?
```

traduzione Elimina un dettaglio scontrino tramite il suo ID.

```
DELETE FROM dieta WHERE id_specie = ? AND id_tipo_cibo = ?
```

traduzione Elimina la voce di dieta di una specie per un determinato tipo di cibo (chiave composta).

```
DELETE FROM dipendente WHERE id_dipendente = ?
```

traduzione Elimina un dipendente tramite il suo ID.

```
DELETE FROM famiglia_specie WHERE id_famiglia_specie = ?
```

traduzione Elimina una famiglia tassonomica tramite il suo ID.

```
DELETE FROM fornitore WHERE id_fornitore = ?
```

traduzione Elimina un fornitore tramite il suo ID.

```
DELETE FROM fornitore_cibo WHERE id_fornitore = ? AND id_tipo_cibo = ?
```

traduzione Elimina l'associazione tra un fornitore e un tipo di cibo (chiave composta).

```
DELETE FROM habitat WHERE id_habitat = ?
```

traduzione Elimina un habitat tramite il suo ID.

```
DELETE FROM mansione WHERE id_mansione = ?
```

traduzione Elimina una mansione tramite il suo ID.

```
DELETE FROM ordine_giornaliero WHERE id_ordine = ?
```

traduzione Elimina un ordine giornaliero tramite il suo ID.

```
DELETE FROM recinto WHERE id_recinto = ?
```

traduzione Elimina un recinto tramite il suo ID.

```
DELETE FROM scontrino WHERE id_scontrino = ?
```

traduzione Elimina uno scontrino tramite il suo ID.

```
DELETE FROM specie WHERE id_specie = ?
```

traduzione Elimina una specie animale tramite il suo ID.

```
DELETE FROM stato_esistenza WHERE id_stato = ?
```

traduzione Elimina uno stato di esistenza tramite il suo ID.

```
DELETE FROM storico_collocazione WHERE id_storico = ?
```

traduzione Elimina una voce dello storico collocazioni tramite il suo ID.

```
DELETE FROM storico_stipendio WHERE id_storico = ?
```

traduzione Elimina una voce dello storico stipendi tramite il suo ID.

```
DELETE FROM tipo_area WHERE id_tipo_area = ?
```

traduzione Elimina un tipo di area tramite il suo ID.

```
DELETE FROM tipo_biglietto WHERE id_biglietto = ?
```

traduzione Elimina un tipo di biglietto tramite il suo ID.

```
DELETE FROM tipo_cibo WHERE id_tipo_cibo = ?
```

traduzione Elimina un tipo di cibo tramite il suo ID.

```
DELETE FROM tipo_fornitura WHERE id_tipo_fornitura = ?
```

traduzione Elimina un tipo di fornitura tramite il suo ID.

```
DELETE FROM tipo_recinto WHERE id_tipo_recinto = ?
```

traduzione Elimina un tipo di recinto tramite il suo ID.

```
DELETE FROM transazione WHERE id_transazione = ?
```

traduzione Elimina una transazione finanziaria tramite il suo ID.

```
DELETE FROM turno WHERE id_turno = ?
```

traduzione Elimina un turno lavorativo tramite il suo ID.

```
DELETE FROM utente WHERE id_utente = ?
```

traduzione Elimina un utente tramite il suo ID.

```
DELETE FROM visita_medica WHERE id_visita = ?
```

traduzione Elimina una visita medica tramite il suo ID.

Chapter 4

Progettazione dell'applicazione

4.1 Architettura dell'applicazione realizzata

L'applicazione è stata sviluppata in Java con JavaFX e segue una struttura MVC piuttosto netta:

- **View:** definiscono la GUI, le tabelle e i moduli di inserimento;
- **Controller:** collegano gli eventi dell'interfaccia alle operazioni di dominio;
- **Model:** contiene le classi entità e i DAO per l'accesso ai dati;
- **Database:** PostgreSQL eseguito tramite Docker.

Le principali scelte architetturali sono:

- **DAO JDBC:** ogni tabella principale ha il proprio DAO, con metodi per CRUD e ricerche mirate;
- **Password hashing:** le password sono memorizzate come hash bcrypt;
- **Configurazione esterna:** i parametri del DB sono letti da variabili d'ambiente;
- **Navigazione centralizzata:** una finestra principale ospita pannelli diversi in base alla sezione selezionata;
- **Controllo dei ruoli:** il pannello di gestione filtra i tab in base al ruolo dell'utente autenticato.

Struttura funzionale dell'interfaccia

L'interfaccia pubblica comprende:

- Home;
- Animali;
- Biglietti;
- Donazioni;
- Login.

L'area riservata include i moduli di amministrazione del sistema. La visibilità delle schermate e delle funzionalità disponibili varia in base al ruolo svolto dall'utente all'interno del parco: non tutti gli utenti hanno accesso a tutti i moduli, ma soltanto a quelli pertinenti alle proprie mansioni. Sono previsti quattro profili utente con differenti permessi e livelli di accesso:

- Amministratore (admin@zoopark.it), password: password123
- Operatore di biglietteria (biglietteria@zoopark.it), password: password123
- Veterinario (vet.rossi@zoopark.it), password: password123
- Guardiano (guardiano@zoopark.it), password: password123

L'utente amministratore dispone di accesso completo a tutte le schermate e a tutte le funzionalità del sistema, mentre gli altri utenti visualizzano esclusivamente le sezioni necessarie allo svolgimento delle proprie attività. Per semplicità e scopi didattici è stata assegnata la stessa password a tutti gli utenti e nel database viene salvata l'hash della password.

Questo è l'elenco di tutte le schermate esistenti:

- saldo e statistiche;
- tipi biglietto;
- ordini giornalieri;
- visite mediche;
- turni;
- personale;
- spese;
- animali;
- classificazione;
- aree;
- recinti.

4.2 Screenshot dell'interfaccia utente

Le figure seguenti sono predisposte per accogliere alcuni screenshot dell'applicazione realizzata per aiutare l'utente all'uso dell'applicativo.

4.2.1 Schermata principale

La schermata principale rappresenta la homepage pubblica del sistema ZooPark. Da questa pagina l'utente può accedere alle informazioni generali del parco, consultare i servizi disponibili e raggiungere l'area riservata tramite l'apposita funzione di login.

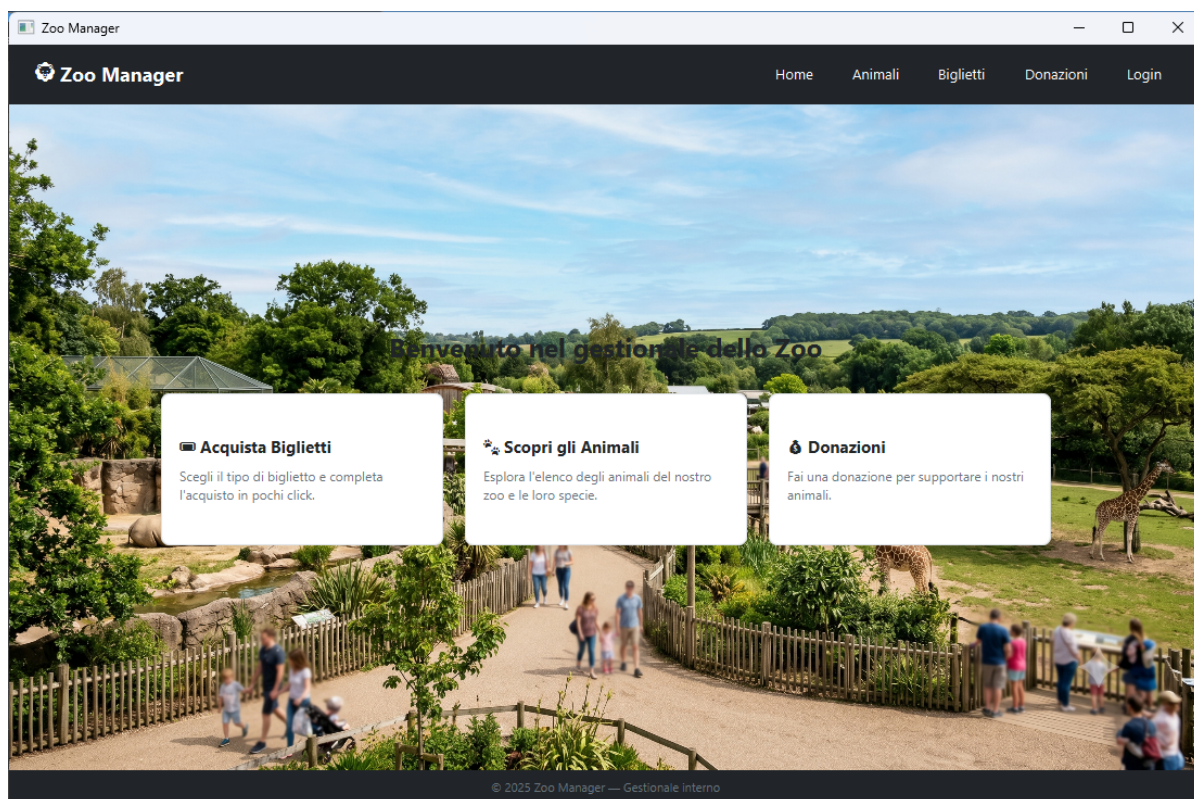


Figure 4.1: Immagine della schermata principale

4.2.2 Schermata di login

La schermata di login consente agli utenti autorizzati di autenticarsi inserendo le proprie credenziali. In base al ruolo associato all'account, il sistema mostrerà funzionalità e sezioni differenti dell'area riservata.

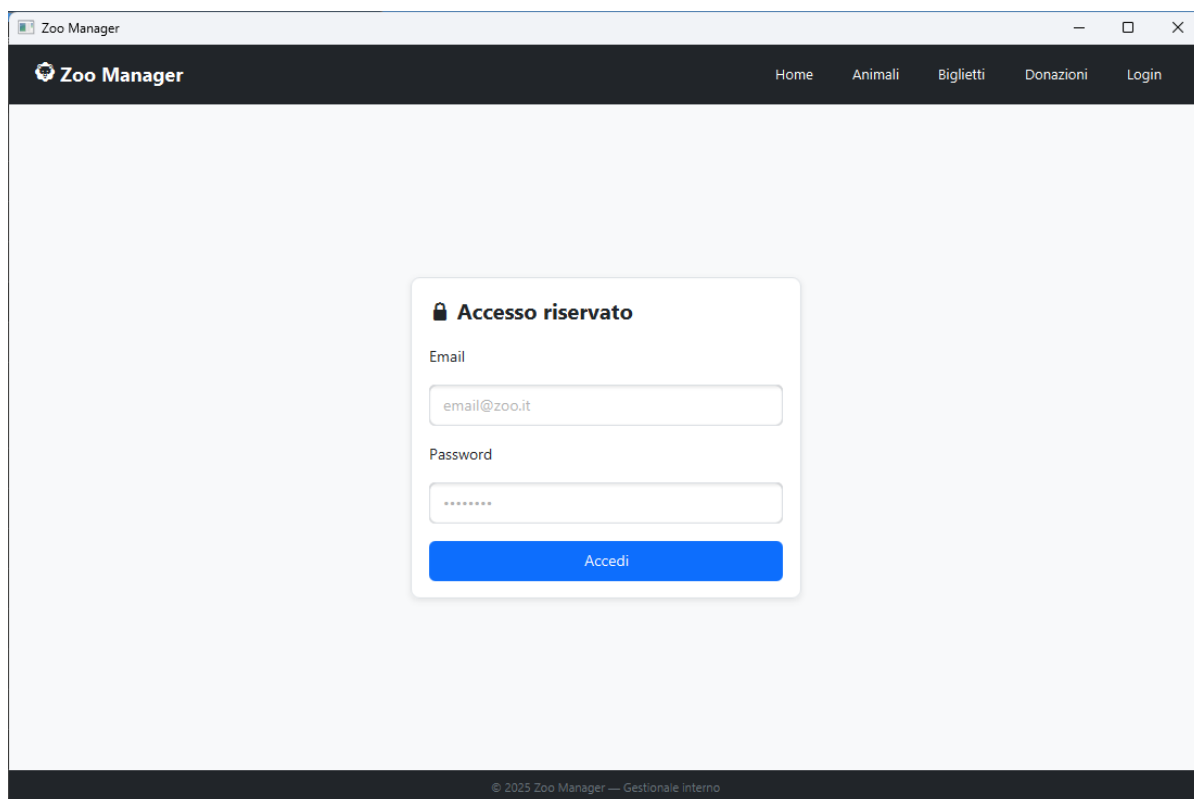


Figure 4.2: Immagine della schermata di login

4.2.3 Schermata dal punto di vista di admin

Una volta effettuato l'accesso come amministratore, vengono rese disponibili tutte le funzionalità del sistema. Questo ruolo dispone di accesso completo ai moduli di gestione degli animali, del personale, delle strutture e delle attività amministrative del parco.

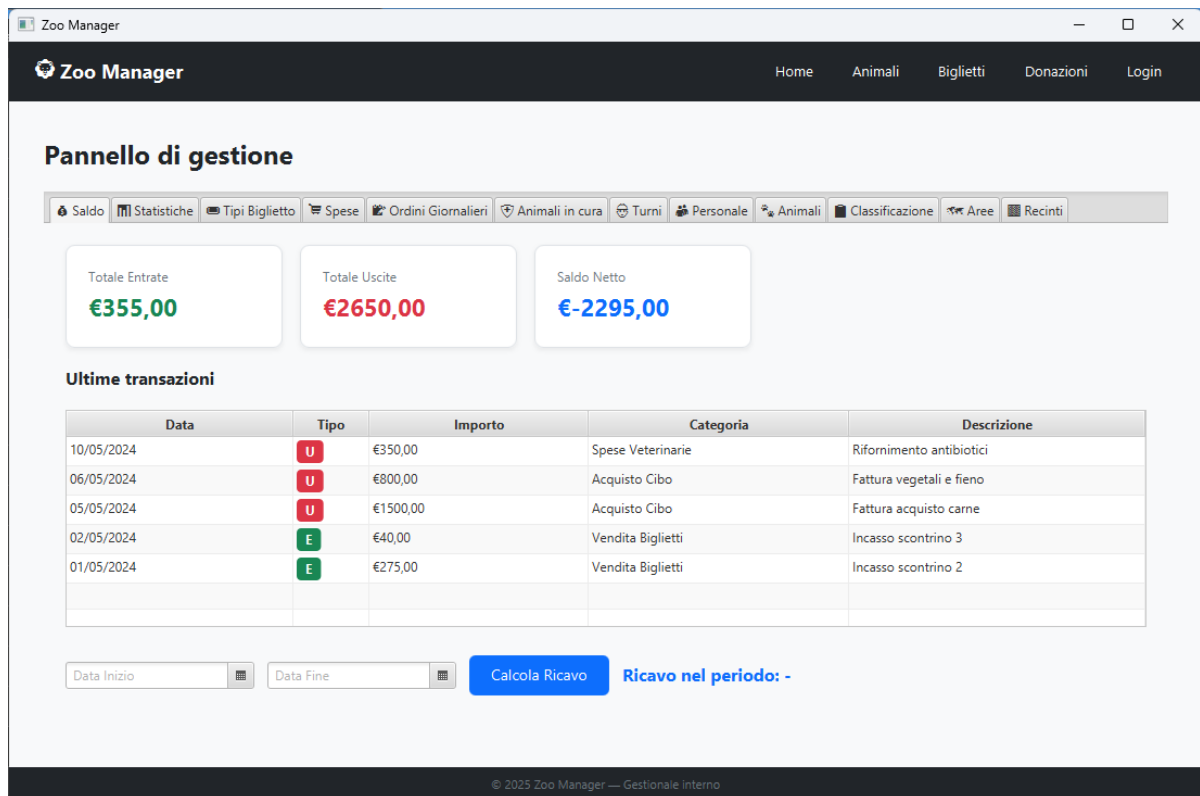


Figure 4.3: Immagine della schermata di logged come admin

4.2.4 Schermata dal punto di vista del veterinario

La schermata visualizzata dal veterinario mostra esclusivamente le funzionalità pertinenti alle sue mansioni. In particolare, il veterinario può consultare e aggiornare le informazioni sanitarie degli animali, senza accedere ai moduli amministrativi riservati ad altri ruoli.

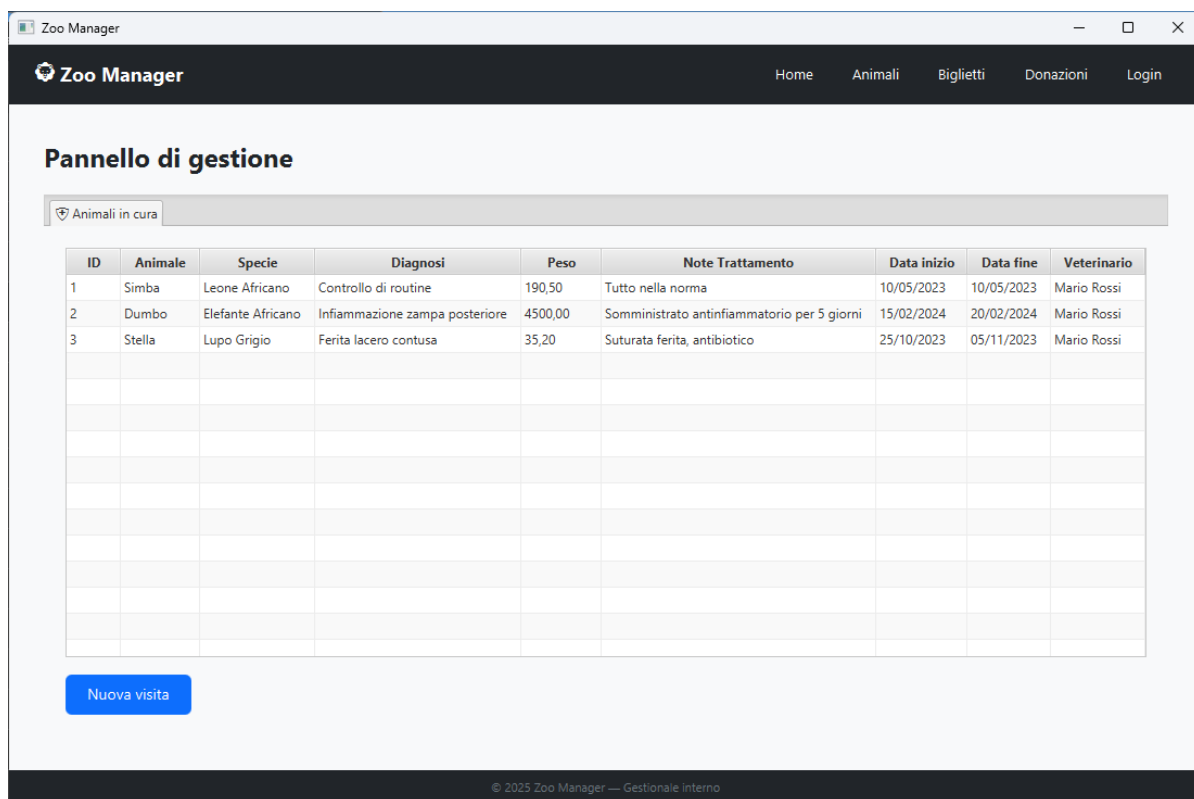


Figure 4.4: Immagine della schermata di logged come veterinario

4.2.5 Schermata dal punto di modifica di un animale (Admin)

Questa schermata consente all'amministratore di modificare i dati relativi a un animale presente nel sistema. Attraverso l'apposito modulo è possibile aggiornare informazioni anagrafiche, caratteristiche, habitat e altri dati gestionali associati all'animale selezionato.

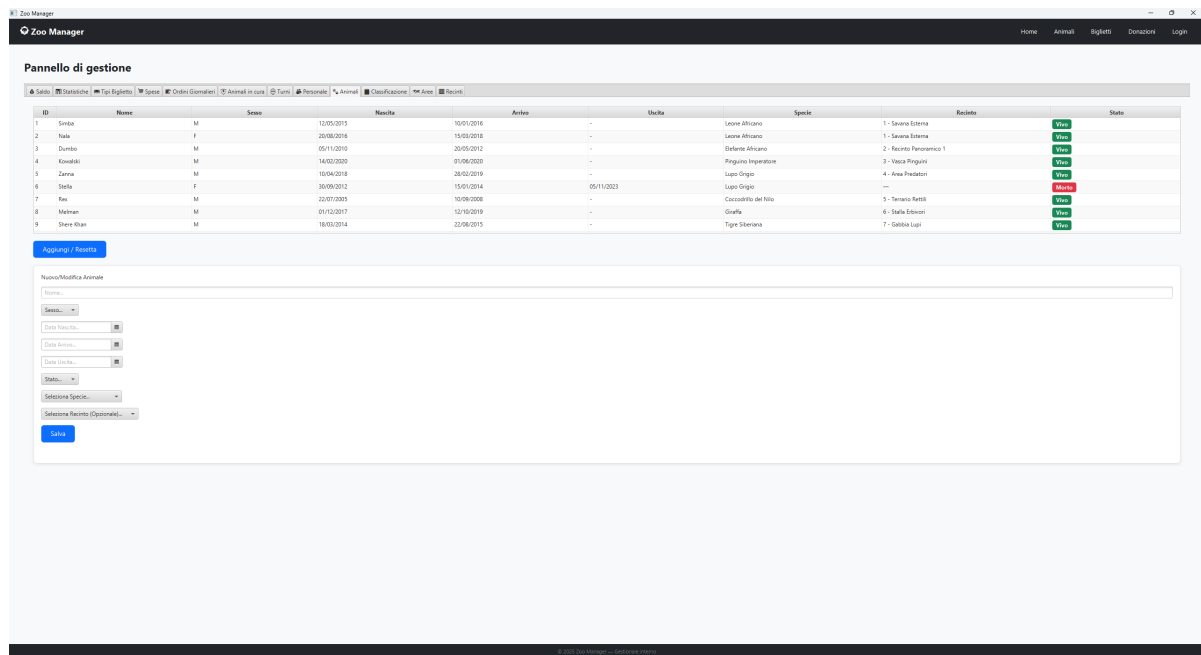


Figure 4.5: Immagine della schermata di modifica di un animale logged come Admin